

# A Battleship Implementation for Synchronous Combat with Dual Microprocessors

Matthew Maclean - 202311912, Omar Elfeki - 202211470, Sam Ruttgaizer - 202312489,  
Steven Dyer - 202227070

**Abstract**—This paper presents the design and implementation of a dual-microprocessor battleship game utilizing two Arduino microcontrollers connected through serial communication. The system features a scaled-down best-of-3 battleship implementation using small LED grids, joystick-based ship selection and targeting, and real-time communication between controllers for synchronized gameplay. Key hardware components include LEDs for visual feedback, numeric displays for score tracking, audio devices for sound cues, and custom logic circuits for display control. The game employs a sophisticated state machine architecture managing ship selection, shooting phases, and game reset protocols with robust message filtering to prevent communication errors. The final implementation demonstrates effective microprocessor coordination, real-time communication, and integrated hardware-software design.

## I. INTRODUCTION AND PROJECT DEFINITION

**B**ATTLESHIP is a classic strategic game where two players are given a grid to tactically place their ships and take turns guessing where their opponents' ships are through requesting shots at a chosen point on the grid. The player is then given some form of indication of the status of their shot at the chosen grid location. When a player has sunk all their opponents' ships, they win the game. This report details an implementation of Battleship utilizing two Arduinos designed to be physical controllers for each player.

This Battleship game has been designed to follow all requirements specified in the ECE4510 Project Brief document [1], and uses two Arduinos connected to multiple peripherals handling control, visual and audio feedback, and communication, as well as logic integrated circuits (ICs) to achieve the desired functionality.

## II. GAME DESIGN

### A. Game Requirements

The final implementation of the battleship game uses the following hardware components:

- 1) 2 Smraza Arduino microcontrollers (main control flow and communication)
- 2) 2 Joysticks (ship/shot selection)
- 3) 2 Buttons (ship/shot confirmation)
- 4) 18 LEDs (battleship grid)
- 5) 2 Passive Buzzers (audio cues)
- 6) 2 7-Segment displays (score/state tracking)
- 7) 2 74HC08 Quad 2-Input AND gates
- 8) 2 74HC32 Quad 2- Input OR gates
- 9) 2 74HC04 Hex Inverters
- 10) 2 9V Batteries (powering both controllers)

- 11) 6 680Ω and 4 220Ω current limiting resistors (for peripherals)

This selection of hardware meets the requirements for inputs and outputs as specified in the project brief document [1]. In determine what hardware could be used for the game, the Arduino UNO datasheet [2] was studied and the design was planned keeping in mind the available pins and feature set of the board.

### B. Control Flow

Unlike the classic game of Battleship, this scaled-down implementation uses one 3x3 grid consisting of LEDs for both ship selection as well as shot targeting. Each player will interact with a joystick to select an LED on the grid and a button to confirm their choice of LED. This process will be the same for both ship selection as well as shot targeting. The game is in a best-of-3 format, with the first player to reach 2 hits winning.

To begin the game, both players are initially in the ship selection phase and can select a singular LED as the location of their ship; if one player selects before the other, they will have to wait until both ships are selected. Once both players have selected a ship, one Arduino, designated by the `isPlayerOne` flag in the program, will move into the shooting state, while the other moves into the idle state, where it will wait for a message from the other Arduino.

When a player is in the shooting phase, there are two options: selecting a shot or resetting the game. If the player chooses to reset the game, achieved by holding down the button for 3 seconds, a message will be sent to the other Arduino, requesting a reset. If a confirmation is sent back, both players will move back into the simultaneous ship selection phase with any previous scores reset to 0.

If a player chooses a target LED to shoot, on button confirmation, the target information will be sent to the other Arduino, where it will return a HIT or MISS code. If a MISS code is returned, the other Arduino moves into the shooting phase while the shooter switches to idle. If a HIT command is returned, the Arduino that returns this command will briefly wait for a `GAME_WON` response from the shooting Arduino, which will send that message if their score reaches 2 from the previous hit. If `GAME_WON` is received by the other Arduino, it will reset, with the other player resetting as well, because their score has reached 2 points, prompting a new game cycle.

If the Arduino receives a HIT command, and the `GAME_WON` signal is not sent back, the game continues with both players moving back into the ship selection phase

to choose a new ship location. The player who got the hit will have a winner's advantage, shooting first again when the next round begins.

The overall control flow of the program is well documented in the sketch code in Appendix A.

### C. Ship and Target Selection

To allow responsive control of the joystick, there are some states that need to be considered to allow a valid joystick movement. Firstly, the joystick must be centered prior to a movement. Secondly, the joystick's current position must be larger than the specified threshold (distance from the center). Once these conditions are met, a move in either the x or y direction will be processed. This implementation processes a move as soon as the joystick surpasses the threshold specified, meaning no perceivable delay in the player's interaction and visual updating of the matrix. Forcing the joystick to return to the center after each move prevents multiple movements from being triggered by one joystick move and allows for precision control over which LED is selected. The lesson provided by Smraza was used as a reference to implement the joystick [3].

In order to allow smooth selection and cursor tracking along the matrix, the digital Arduino pins used for the LEDs are stored in a 3x3 array named LEDGRID, with each pin number in the array corresponding to its physical location on the grid. To track which LED is currently being selected by the joystick, another 2-element array with the variable name GridLocation is used to store the x and y coordinates of the joystick on the grid. Using the first and second elements of GridLocation, the digital pin corresponding to that location can be accessed within the matrix and controlled using the digitalWrite command. To allow the player to distinguish which LED is selected, it will blink at 250-ms intervals, controlled in the main loop of the program.

In the shot selection phase, grid locations that have already been selected as shots will stay illuminated to give a visual reminder to the player. Another array of size 9 stores either a 1 or 0 at each element corresponding to whether that LED has already been targeted. When the player moves back to their shot selection turn after the other player misses, this list is iterated through, turning on each LED where a 1 is stored. If a player tries to reselect a chosen LED, their cursor blinks rapidly, no message is sent to the other Arduino, and the player is allowed to select another target.

As previously mentioned, a button press confirms the ship/shot selection, which enters the main control flow of the program for communication between Arduinos. Wiring of the button was based on a lesson provided by Smraza [4].

### D. 7-Segment Display: Score and State Tracking

One of the output devices utilized in the game is the 7-Segment Display, used to display the player's score, as well as show the player if they are in the shooting or selection phase. The 7-Segment Display takes two digital pins on the Arduino as outputs and uses the 4 possible binary combinations that can be created as inputs to light the desired segments on the display.

The four patterns that can be output from the 7-Segment display are 0, 1, 2, and S; the numbers are an indication of a player's given score, only displayed while the Arduino is in an idle state (waiting for a message). When a player is in the shooting or selection phase, the output of the 7-Segment Display is an 'S' pattern.

The 2 digital pins on the Arduino labeled A and B (MSB and LSB, respectively) are passed as a 2-bit binary input into a logic circuit that handles the combinational logic required to illuminate the needed patterns. Table I shows what binary input corresponds to what pattern on the 7-Segment display. Boolean algebra rules were used to simplify the wiring of each segment (segments wired from the pinout provided from the display datasheet [5]) and reduce the required number of logic gates.

TABLE I  
BINARY INPUTS CORRESPONDING TO 7-SEGMENT OUTPUTS

|    |   |
|----|---|
| 00 | 0 |
| 01 | 1 |
| 10 | 2 |
| 11 | S |

Overall, for each controller, 2 AND gates were used on a 74HC08 IC [6], 4 OR gates on a 74HC32 IC [7], and 2 NOT gates on a 74HC04 [8]. The complete wiring diagram can be seen in Figure 1, while the Boolean logic simplification can be seen in Appendix B.

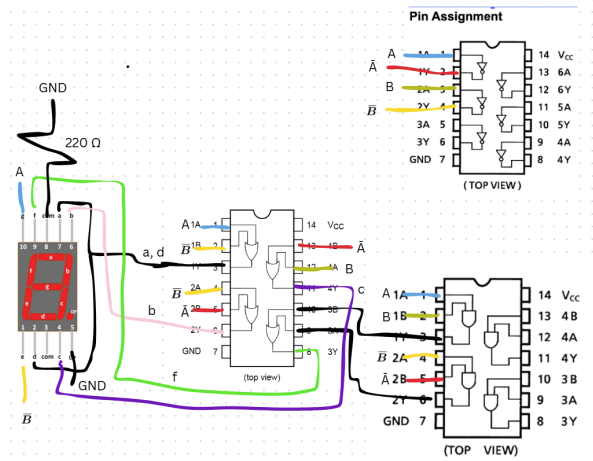


Fig. 1. Simplified Logic Circuit Wiring Diagram for 7-Segment Display

### E. Multi-Arduino Communication

Because two Arduinos were used due to the physical pin limitations on each microcontroller, robust communication between them is required to track the overall game state. Connecting the Arduinos are 5 lines:

- 1) Common ground
- 2) Arduino digital output to one input of an AND comparator for ship selection confirmation
- 3) Output line from the AND comparator back to an input pin on the Arduino (logic HIGH only when both players have selected their ships)

- 4) Serial output line
- 5) Serial input line

There are multiple communication protocols that were considered when implementing communication between the Arduinos, namely inter-integrated circuit (I2C), serial-to-peripheral interface (SPI), and universal asynchronous receiver transmitter (UART). UART was chosen for a number of reasons. First, the Smraza development board inherently supports external UART communication via the receiver (RX) and transmitter (TX) pins on the board. Using I2C or SPI would mean that analog and digital pins would have to be used for communication, not feasible in the pin-limited design. Second, I2C and SPI both implement a controller and target configuration for the Arduinos, meaning one Arduino would send commands to the other, and the other responds to the commands of the controller Arduino, meaning large differences in the main programs flashed to each board, leading to higher complexity in game state synchronization. The external libraries required by these communication protocols further add to the complexity. UART communication does not use a controller and target communication naturally, and uses functions in the preexisting Arduino IDE without any external libraries.

UART communication was set up on each Arduino by connecting the RX and TX pins of one Arduino to the opposite pins of the other (RX connects to TX, TX connects to RX). To read and transmit data between Arduinos, the `Serial.read()` and `Serial.println()` functions were used. Because UART does not use a clock to synchronize the Arduinos, the baud rate is used to synchronize data transmission and collection, which was set to 9600bps for both Arduinos.

To keep communication as simple as possible, only integer values are sent on the TX lines. There are 14 possible codes that are sent between Arduinos; the codes and their corresponding meaning can be seen in Table II.

TABLE II  
ACCEPTED INTEGER VALUES AND THEIR MEANING

|      |                                        |
|------|----------------------------------------|
| 0    | MISS                                   |
| 1    | HIT                                    |
| 2-10 | LED digital pin that has been targeted |
| 97   | RESET_CONFIRM                          |
| 98   | GAME_WON                               |
| 99   | RESET_REQUEST                          |

#### F. Visual and Audio Cues

To indicate to each player whether a shot is a hit or a miss, visual patterns on the LED matrix, as well as audio chimes using a buzzer, were created. On every shot selection, that player's 3x3 grid will illuminate each row in ascending order to replicate a radar-scanning scenario where the shot is checked if it is a hit or miss. The scan pattern is repeated multiple times and is then followed by one of two patterns that are displayed on both players' grids. For a hit, a circular, expanding explosion pattern flashes as seen in Figure 2, indicating the end of a round. On a miss, a blinking X pattern is illuminated as shown in Figure 3, indicating to both players that the shot was unsuccessful and the game continues.

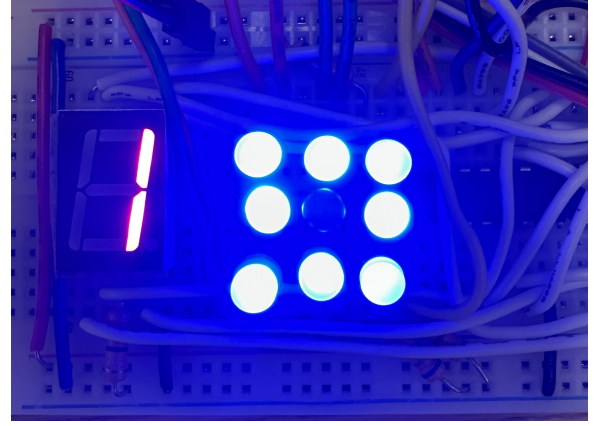


Fig. 2. Hit Pattern Displayed on LED Matrix

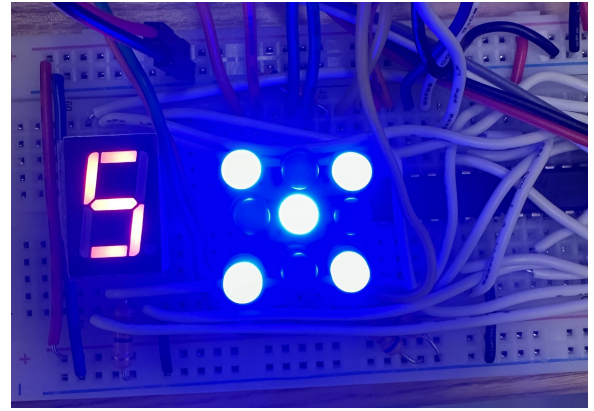


Fig. 3. X Pattern Displayed on LED Matrix

Another feature implemented was audio chimes using a buzzer on each player's controller that plays either a positive or negative chime while the hit/miss patterns are being displayed, giving each player another cue for recognizing game state. Examples from a Smraza lesson on buzzer control [9] were used as inspiration for the design.

### III. TESTING PROTOCOLS

#### A. Subsystem Testing

The importance of subsystem testing can not be overstated. By testing a smaller portion of the overall system, diagnosing and troubleshooting issues in smaller parts of the implementation can be completed without spending large amounts of time attempting to find the same issue in a larger, full system test. The wiring diagrams for each subsystem are found in Appendix B.

When constructing the LED grid, to check the wiring and connections to the Arduino, patterns were printed onto the grid to confirm the desired output. The joystick and button were tested by having a simplified main program loop where only the joystick movement and button presses were handled, confirming cursor functionality as well as shot tracking. On a button press, the LED in the grid will stay lit up to test the storage of previously chosen shots.

To ensure the correct integer commands were being sent via UART, our group used the Arduino IDE serial monitor to observe messaging.

### B. Dummy Arduino Testing

To test the ability to send a receive messages over the UART RX and TX pins, a dummy Arduino was set up using a simple program where it would have a predetermined ship selected and it would wait to receive messages from the main Arduino to compare with this selection. It would then return a HIT or MISS code, which was handled by the main Arduino. The sketch code for the test Arduino can be found in Appendix C.

It was confirmed that Arduino communication was successful when on a miss, the desired X pattern would display, and on a hit, the desired pattern would also display, with the 7-segment score being incremented.

### C. Full System Integration

For testing the main control flow and the overall implementation, both fully wired controllers were connected with the aforementioned harnessing. In this stage, every possible game scenario was written in a list and tested. During this testing phase, any circuitry or control flow errors were noted and resolved. Any lingering issues that had no quick solution had their potential reasons behind the error recorded and checked individually. The tests were then again conducted after each iteration of bug fixing, repeating until the state of the game was in working order. Figures of the full implementation can be found in Appendix D.

## IV. TROUBLESHOOTING AND RESULTS

### A. UART Handshaking and Message Filtering

Initially, when an Arduino was in an idle state and waiting for a message, it would simply wait until any serial data was available to be read from the buffer, and then would try to process that data. The issue with this is that if there were any noise or interference on the UART lines, erroneous data could be read and then attempted to be processed by the Arduino, causing errors in game synchronization.

To fix this problem, when the Arduino goes into its idle state, it will read any incoming data on the UART line, but only move into data handling if an expected message is read, ignoring all other data. This ensures that the idling Arduino will only process a correct integer code that is sent, keeping the game synchronized.

There were also many issues trying to handle simultaneous game resetting on both Arduinos. The original implementation used just one message to the other Arduino, telling it to reset. But this caused timing problems because the Arduino that sent the message to reset would not know if the other Arduino had correctly processed the message.

To get around this, once an Arduino sends a message to the other requesting a reset, it will then wait for an acknowledgement message to process its own reset protocol. This ensures both Arduinos reset at the same time to move into a new game loop.

### B. Low Battery Voltage Issues

After running the game with the Arduinos being powered by the 9V batteries provided by the Smraza kit, their voltages decreased over a long period of testing, and nondeterministic bugs in the game loop arose, such as the LED matrices randomly flashing in different patterns, buzzer chimes not playing, as well as breaking the control flow of the game. After reading the battery voltages using a multimeter, it was found that both were providing about 6V, compared to their labelled 9V.

To confirm the batteries were causing this issue, a control test of the game using laptops to power both Arduinos was performed. Once it was found that these issues did not persist when getting the required power from a laptop, the 9V batteries were replaced for both player controllers.

## V. CONCLUSION

To conclude, a Battleship game was designed and constructed utilizing two Arduinos connected with a UART communication protocol, various hardware components, and programmed software. The final game design involved the use of both hardware and software logic systems, as well as multiple input and output devices. After testing using an iterative method and troubleshooting timing and communication issues, the final Battleship game reached the desired functionality and met all requirements.

## REFERENCES

- [1] A. Noel, *ECE4510 Project Brief 2024-2025*, ECE-4510, Memorial University. [Online]. Available: <https://online.mun.ca/d2l/le/content/621911/viewContent/5710132/View>
- [2] Arduino, "Arduino® UNO R3", Arduino UNO R3 datasheet, Jul. 2025
- [3] Smraza, "Joystick Test Experiment", Smraza Arduino Uno Kit Joystick datasheet, Mar. 2017
- [4] Smraza, "Button", Smraza Arduino Uno Kit Button datasheet, Mar. 2017
- [5] LITE-ON Technology Corp., "LED Display Product Data Sheet LSHD-5503" LSHD-5503 datasheet, Jul. 2010
- [6] Toshiba, "TOSHIBA CMOS Digital Integrated Circuit Silicon Monolithic TC74HC08AP, TC74HC08AF" TC74HC08AP/AF datasheet, Mar. 2014
- [7] Toshiba, "TOSHIBA CMOS Digital Integrated Circuit Silicon Monolithic TC74AC32P, TC74AC32F, TC74AC32FN, TC74AC32FT" TC74AC32P/F/FN/FT datasheet, Oct. 2007
- [8] Toshiba, "TOSHIBA CMOS Digital Integrated Circuit Silicon Monolithic TC74HC04AP, TC74HC04AF, TC74HC04AFN" TC74HC04AP/AF/AFN datasheet, Oct. 2007
- [9] Smraza, "Passive Buzzer", Smraza Arduino Uno Kit Passive Buzzer datasheet, Mar. 2017

## APPENDIX A MAIN ARDUINO SKETCH

```

// set to false for Player 2
bool myTurn = true;
// set to false for Player 2
// determines who goes first after reset
bool isPlayerOne = true;

// values corresponding to joystick position
const int LEFT = 0;
const int RIGHT = 1;
const int UP = 2;
const int DOWN = 3;
const int CENTER = 4;

// labeled pins for 7-segment display
const int pinA = 13;
const int pinB = 12;

// pin designation for joystick and button
const int xpotPin = A0;
const int ypotPin = A1;
const int buttonPin = A4;

const int buzzerPWM = 11;

// threshold value to constitute valid joystick move
const int THRESHOLD = 200;
const int ORIGIN = 512;

// reset confirmation protocol constants
const int RESET_REQUEST = 99;      // request to reset
const int RESET_CONFIRM = 97;      // confirmation that reset is ready
const int GAME_WON = 98;           // game won signal

int joystickPos = CENTER;           // stores the joystick position
bool wasLastMoveCentered = true;    // last move centered
bool validMovementDetected = false; // bool storing if a valid move is detected
int buttonVal;                      // initializing button value variable
int newPosition;                    // holds new position of joystick

// pin to input of AND gate to confirm selection
const int readyPin = A2;
// pin from output of AND gate to check if both players have selected
const int readyCheck = A3;

// matrix storing the digital pins the LEDs are connected to (in correct order)
const int LEDGRID [3][3] = {{2, 5, 8}, {3, 6, 9}, {4, 7, 10}};

// used to get value in LEDGRID
int GridLocation[2] = {0, 0};

// determine what LED was last selected
int lastOnLED = LEDGRID[0][0];
int myShipPos;
int myScore = 0;

```

```

// list storing state of all LEDs ON/OFF
int selectedList[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
int listMap = lastOnLED - 2;

// flag for shot selection phase
bool selectionPhase = true;
bool shootingPhase = false;
bool isInitialSelection = true; // true for game start/reset, false post-hit selection
bool gameResetPending = false; // flag to indicate a reset is in progress
bool displayPatternActive = false; // flag to disable display control during patterns
int hitmiss;

int receivedTarget; // value received from other arduino VIA UART

// variables for handling LED blinking
unsigned long lastBlinkTime = 0;
bool blinkState = true;
const unsigned long BLINK_INTERVAL = 250;
unsigned long currentTime;
i
// function prototypes
bool isJoystickCentered();
int getJoystickDirection();
void handleMovement(int direction);
void updateLEDs();
void LEDON (int LEDNumber);
void LEDOFF (int LEDNumber);
void XPattern();
void ShotPattern();
void HitPattern();
void display0();
void display1();
void display2();
void displayS();
void displayScore(int score);
void allLEDOFF();
int readTarget();
void sendTarget(int target);
void performReset();

void setup()
{
    // initialize all used pins
    pinMode(xpotPin, INPUT);
    pinMode(ypotPin, INPUT);
    Serial.begin(9600); // baud rate, UART
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
    pinMode(6, OUTPUT);
    pinMode(7, OUTPUT);
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
    pinMode(buttonPin, INPUT);

```

```
pinMode(readyPin, OUTPUT);
pinMode(readyCheck, INPUT);
pinMode(pinA, OUTPUT);
pinMode(pinB, OUTPUT);
displayS();
}

void loop()
{
    // check if a reset is pending, execute it immediately and skip everything else
    if(gameResetPending)
    {
        performReset();
        gameResetPending = false;
        return;
    }

    newPosition = getJoystickDirection();

    buttonVal = digitalRead(buttonPin);

    if(buttonVal == HIGH)
    {
        // check for 3 second button press for game reset
        unsigned long buttonPressStart = millis();
        bool resetTriggered = false;

        while(digitalRead(buttonPin) == HIGH)
        {
            if(millis() - buttonPressStart >= 3000)
            {
                // start reset confirmation process
                sendTarget(RESET_REQUEST);

                // wait for confirmation from opponent
                int response = 0;
                unsigned long waitStart = millis();
                while(millis() - waitStart < 5000) // wait up to 5 seconds for confirmation
                {
                    if(Serial.available())
                    {
                        response = readTarget();
                        if(response == RESET_CONFIRM)
                        {
                            // opponent confirmed, set flag to reset on next loop iteration
                            gameResetPending = true;
                            resetTriggered = true;
                            break;
                        }
                    }
                }
                delay(10);
            }

            // if no confirmation received, cancel reset
            if(!resetTriggered)
            {
                // flash LEDs to indicate reset cancelled
                for(int i = 0; i < 3; i++)
```

```

    {
        allLEDOFF();
        delay(200);
        for(int j = 2; j <= 10; j++)
        {
            LEDON(j);
        }
        delay(200);
    }
    allLEDOFF();
}

// wait for button release
while(digitalRead(buttonPin) == HIGH)
{
    delay(10);
}
return; // exit early
}
delay(10);
}

// only process normal button actions if reset was NOT triggered
if (!resetTriggered)
{
    if (shootingPhase && myTurn)
    {
        // check if this target was already selected
        listMap = lastOnLED - 2;
        if (selectedList[listMap] == 1)
        {
            // rapidly for 2 seconds if has been selected
            unsigned long flashStart = millis();
            while (millis() - flashStart < 2000)
            {
                LEDON(lastOnLED);
                delay(100);
                LEDOFF(lastOnLED);
                delay(100);
            }
            // continue shot selection
            return;
        }

        // player turn to shoot at new target
        selectedList[listMap] = 1;
        ShotPattern();

        // send target to opponent
        sendTarget(lastOnLED);

        // wait for hit/miss response
        hitmiss = readTarget();

        // this ensures reset is processed
        if (hitmiss == GAME_WON)
        {
            // set flag to reset on next loop iteration

```

```

    gameResetPending = true;
    return; // exit early, reset will happen next loop
}

if (hitmiss == 1)
{
    // increment score and reset both players to selection phase
    myScore++;
    displayScore(myScore);
    HitPattern();

    // check if player reached 2 points (game won)
    if(myScore >= 2)
    {
        // send GAME_WON signal to opponent to reset
        sendTarget(GAME_WON);
        delay(100);
        gameResetPending = true; // set flag to reset on next loop iteration
        return; // exit early reset will happen next loop
    }

    // clear selected lists and reset to selection phase
    for(int i = 0; i < 9; i++)
    {
        selectedList[i] = 0;
    }

    selectionPhase = true;
    shootingPhase = false;
    isInitialSelection = true; // both players need to select new ships
    myTurn = true; // winner goes first next round

    delay(2000);
    allLEDOFF();

    // reset cursor to top-left
    GridLocation[0] = 0;
    GridLocation[1] = 0;
    lastOnLED = LEDGRID[0][0];
}

else if (hitmiss == 0)
{
    // display X pattern then switch turns
    XPattern();
    displayScore(myScore);
    myTurn = false;
    delay(1000);
    allLEDOFF();
}
}

else if (selectionPhase && isInitialSelection)
{
    // ship selection phase - both players select ships if isInitialSelection=true
    LEDON(lastOnLED);
    myShipPos = lastOnLED;
    digitalWrite(readyPin, HIGH);
}

```

```

    // wait until both players are ready (AND gate output high)
    while(digitalRead(readyCheck) != HIGH)
    {
        delay(10);
    }
    delay(500);

    digitalWrite(readyPin, LOW);

    // move to shooting phase
    selectionPhase = false;
    shootingPhase = true;
    isInitialSelection = false; // no longer initial selection

    // reset cursor to top-left
    GridLocation[0] = 0;
    GridLocation[1] = 0;
    lastOnLED = LEDGRID[0][0];
}
} // end of normal button action processing

// wait for button release to prevent multiple triggers
while(digitalRead(buttonPin) == HIGH)
{
    delay(10);
}
delay(50); // debounce delay
}

// handle opponent turn
if (shootingPhase && !myTurn)
{
    // wait for opponent shot with validation
    if (Serial.available())
    {
        receivedTarget = 0;
        // wait for valid shot data, reset request, confirm, or game won signal
        while(receivedTarget < 2 ||
            (receivedTarget > 10 &&
                receivedTarget != RESET_REQUEST &&
                receivedTarget != GAME_WON &&
                receivedTarget != RESET_CONFIRM))
        {
            receivedTarget = readTarget();
            delay(10);
        }

        if(receivedTarget == RESET_REQUEST)
        {
            sendTarget(RESET_CONFIRM);
            delay(100);
            gameResetPending = true; // set flag to reset on next loop iteration
            return; // reset will happen next loop
        }

        // check if opponent won the game
        if(receivedTarget == GAME_WON)
        {

```

```

        // set flag to reset on next loop iteration
        gameResetPending = true;
        return; // reset will happen next loop
    }

    if(receivedTarget == myShipPos)
    {
        sendTarget(1); // send hit confirm
        HitPattern();

        // wait to see if opponent sends GAME_WON signal
        delay(300);
        if(Serial.available())
        {
            int followUp = readTarget();
            if(followUp == GAME_WON)
            {
                // set flag to reset on next loop iteration
                gameResetPending = true;
                return; // don't process normal hit logic
            }
        }

        // normal hit processing - opponent didn't win
        for(int i = 0; i < 9; i++)
        {
            selectedList[i] = 0;
        }

        selectionPhase = true;
        shootingPhase = false;
        isInitialSelection = true; // both players need to select new ships
        myTurn = false; // opponent goes first next round

        delay(2000);
        allLEDOFF();

        // reset cursor to top-left
        GridLocation[0] = 0;
        GridLocation[1] = 0;
        lastOnLED = LEDGRID[0][0];
    }
    else
    {
        // send back miss signal
        sendTarget(0);
        XPattern();
        myTurn = true;
        delay(1000);
        allLEDOFF();
    }
}

if (validMovementDetected)
{
    joystickPos = newPosition;
    handleMovement(joystickPos);
}

```

```

    }
    else if (newPosition == CENTER && joystickPos != CENTER)
    {
        joystickPos = CENTER;
    }

    currentTime = millis();
    if (currentTime - lastBlinkTime >= BLINK_INTERVAL)
    {
        blinkState = !blinkState;
        lastBlinkTime = currentTime;
    }

    // 7 segment display control
    if (!displayPatternActive) // only run display control when not showing patterns
    {
        if ((selectionPhase && isInitialSelection) || (shootingPhase && myTurn))
        {
            // show S for selection and shooting
            displayS();
        }
        else
        {
            // show score at all other times
            displayScore(myScore);
        }
    }

    updateLEDs();
    delay(50);
}

// check if joystick is centered
bool isJoystickCentered()
{
    int xValue = analogRead(xpotPin);
    int yValue = analogRead(ypotPin);

    int xDistance = abs(xValue - ORIGIN);
    int yDistance = abs(yValue - ORIGIN);

    return (xDistance < THRESHOLD && yDistance < THRESHOLD);
}

// determine joystick direction
int getJoystickDirection()
{
    // check if joystick is centered
    bool isCentered = isJoystickCentered();
    validMovementDetected = false;

    // if centered return CENTER
    if (isCentered)
    {
        wasLastMoveCentered = true;
        return CENTER;
    }
}

```

```

}

if (wasLastMoveCentered)
{
    // if not centered and last move centered, determine joystick direction
    wasLastMoveCentered = false;
    validMovementDetected = true;

    int xRead = analogRead(xpotPin);
    int yRead = analogRead(ypotPin);
    int xDistance = xRead - ORIGIN;
    int yDistance = yRead - ORIGIN;

    if (abs(xDistance) > abs(yDistance))
    {
        return (xDistance > 0) ? RIGHT : LEFT;
    }
    else
    {
        return (yDistance > 0) ? DOWN : UP;
    }
}

/* if not centered and last move not centered,
 * return last position with validMovementDetected set false
 */
return joystickPos;
}

// move the selected LED on the grid based on joystick input
void handleMovement(int direction)
{
    int currentListIndex = lastOnLED - 2;
    if (selectedList[currentListIndex] == 0)
    {
        LEDOFF(lastOnLED);
    }

    switch(direction)
    {
        case LEFT:
            // check if on left edge and wrap to right edge
            if(GridLocation[1] == 0)
            {
                GridLocation[1] = 2;
            }
            else
            {
                GridLocation[1]--;
            }

            // set cursor to new grid location
            lastOnLED = LEDGRID[GridLocation[0]][GridLocation[1]];
            break;

        case RIGHT:
            // check if on right edge and wrap to left edge
            if(GridLocation[1] == 2)

```

```

    {
        GridLocation[1] = 0;
    }
    else
    {
        GridLocation[1]++;
    }

    lastOnLED = LEDGRID[GridLocation[0]][GridLocation[1]];
    break;

case UP:
    // check if on upper edge and wrap to bottom edge
    if(GridLocation[0] == 0)
    {
        GridLocation[0] = 2;
    }
    else
    {
        GridLocation[0]--;
    }

    lastOnLED = LEDGRID[GridLocation[0]][GridLocation[1]];
    break;

case DOWN:
    // check if on bottom edge and wrap to upper edge
    if(GridLocation[0] == 2)
    {
        GridLocation[0] = 0;
    }
    else
    {
        GridLocation[0]++;
    }

    lastOnLED = LEDGRID[GridLocation[0]][GridLocation[1]];
    break;
}
}

// update status of all LEDs
void updateLEDs()
{
    // temporarily turn off all LEDs
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            LEDOFF(LEDGRID[i][j]);
        }
    }

    // Only show LEDs when it's my turn or during selection phase
    if ((shootingPhase && myTurn) || (selectionPhase && isInitialSelection))
    {
        // turn back on LEDs that are in the selected list (only during shooting phase)
        if (shootingPhase)

```

```

    {
        for (int i = 0; i < 9; i++)
        {
            if (selectedList[i] == 1)
            {
                LEDON(i + 2);
            }
        }
    }

    // turn on blinking for cursor
    if (blinkState)
    {
        LEDON(lastOnLED);
    }
    else
    {
        LEDOFF(lastOnLED);
    }
}

// turn on specified LED
void LEDON (int LEDNumber)
{
    digitalWrite(LEDNumber, HIGH);
}

// turn of specified LED
void LEDOFF (int LEDNumber)
{
    digitalWrite(LEDNumber, LOW);
}

// turn off all LEDs
void allLEDOFF()
{
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            LEDOFF(LEDGRID[i][j]);
        }
    }
}

// display an X on a miss
void XPattern()
{
    tone(buzzerPWM, 400, 750);
    displayPatternActive = true; // 7-segment display control flag
    for (int i = 0; i < 3; i++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                LEDOFF(LEDGRID[i][j]);
            }
        }
    }
}

```

```

    }
}

delay(400);

LEDON(2);
LEDON(8);
LEDON(6);
LEDON(4);
LEDON(10);

delay(400);
}
displayPatternActive = false; // reset flag to allow 7-segment display control again
}

// display incrementing row pattern on each shot
void ShotPattern()
{
    displayPatternActive = true;
    for(int i = 0; i < 3; i++)
    {
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                LEDOFF(LEDGRID[i][j]);
            }
        }

        delay(400);

        for (int i = 0; i < 3; i++)
        {
            LEDON(LEDGRID[2-i][0]);
            LEDON(LEDGRID[2-i][1]);
            LEDON(LEDGRID[2-i][2]);

            delay(300);

            if (i < 2)
            {
                LEDON(LEDGRID[1-i][0]);
                LEDON(LEDGRID[1-i][1]);
                LEDON(LEDGRID[1-i][2]);

                delay(100);
            }

            LEDOFF(LEDGRID[2-i][0]);
            LEDOFF(LEDGRID[2-i][1]);
            LEDOFF(LEDGRID[2-i][2]);
        }
    }
    displayPatternActive = false;
}

// display pixelated explosion on hit

```

```
void HitPattern()
{
    displayPatternActive = true;

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            LEDOFF(LEDGRID[i][j]);
        }
    }

    delay(400);

    tone(buzzerPWM, 523, 400);
    LEDON(6);

    delay(400);

    tone(buzzerPWM, 659, 400);
    LEDON(5);
    LEDON(3);
    LEDON(9);
    LEDON(7);

    delay(400);

    tone(buzzerPWM, 784, 400);
    LEDOFF(6);
    LEDON(2);
    LEDON(8);
    LEDON(4);
    LEDON(10);

    delay(400);
    displayPatternActive = false;
}

// functions for displaying numbers to 7-segment display
void display0()
{
    digitalWrite(pinB, LOW);
    digitalWrite(pinA, LOW);
}

void display1()
{
    digitalWrite(pinB, HIGH);
    digitalWrite(pinA, LOW);
}

void display2()
{
    digitalWrite(pinB, LOW);
    digitalWrite(pinA, HIGH);
}

void displayS()
```

```
{
    // display S on 7-segment indicating selection/shooting
    digitalWrite(pinB, HIGH);
    digitalWrite(pinA, HIGH);
}

void displayScore(int score)
{
    // display value on 7-segment based on score
    switch(score)
    {
        case 0:
            display0();
            break;

        case 1:
            display1();
            break;

        case 2:
            display2();
            break;
    }
}

int readTarget()
{
    int recTarget;

    // wait until there is data
    while(!Serial.available())
    {
    }

    recTarget = Serial.parseInt();

    // clear serial buffer
    while (Serial.available() > 0)
    {
        char c = Serial.read();
        if (c == '\n') break; // stop clearing when newline found
    }

    return recTarget;
}

void sendTarget(int target)
{
    // send integer over UART
    Serial.println(target);
}

void performReset()
{
    // reset game state (score and leds)
    myScore = 0;
    for(int i = 0; i < 9; i++)
    {
```

```

    selectedList[i] = 0;
}

selectionPhase = true;
shootingPhase = false;
isInitialSelection = true;
myTurn = isPlayerOne; // initial Player 1 always goes first after reset
gameResetPending = false;

// flash all LEDs to indicate reset
for(int flash = 0; flash < 5; flash++)
{
    for(int i = 2; i <= 10; i++)
    {
        LEDON(i);
    }
    delay(200);
    allLEDOFF();
    delay(200);
}

// reset cursor to top-left
GridLocation[0] = 0;
GridLocation[1] = 0;
lastOnLED = LEDGRID[0][0];
displayS();
}

```

## APPENDIX B CIRCUIT DIAGRAMS

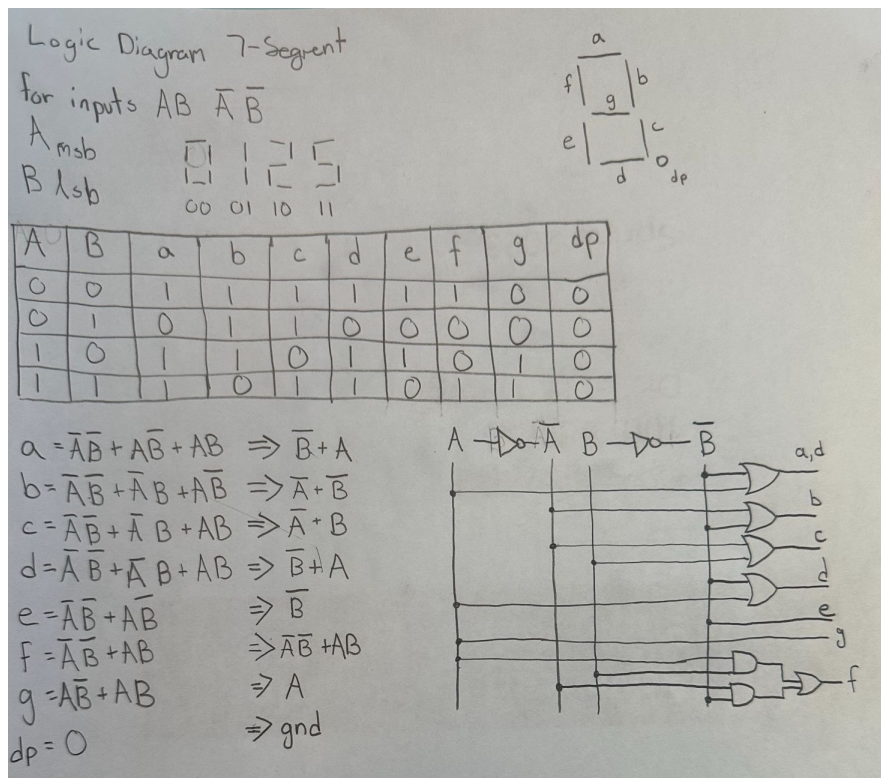


Fig. 4. Boolean Logic Simplification for 4 7-Segment States

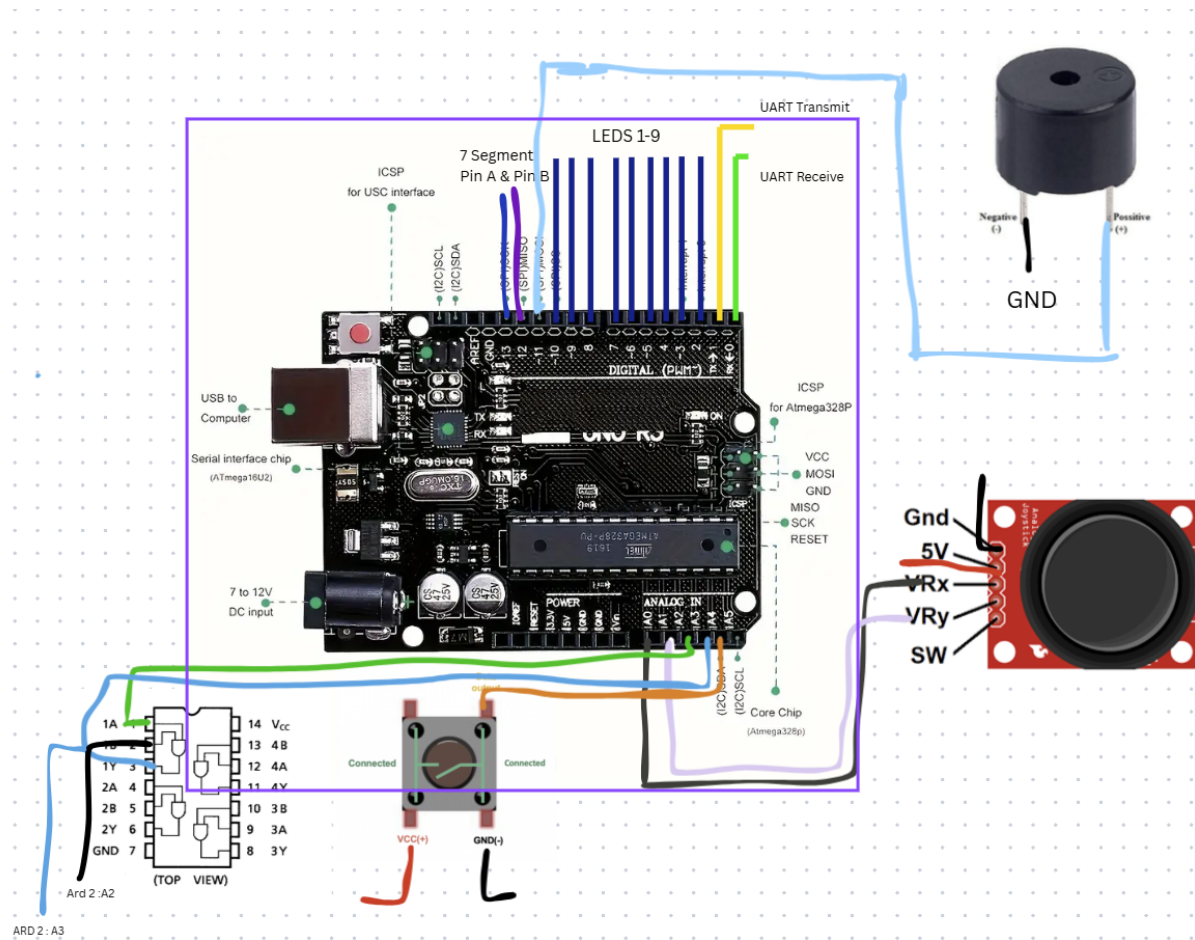


Fig. 5. Wiring diagram for the Buzzer, Joystick, and Button

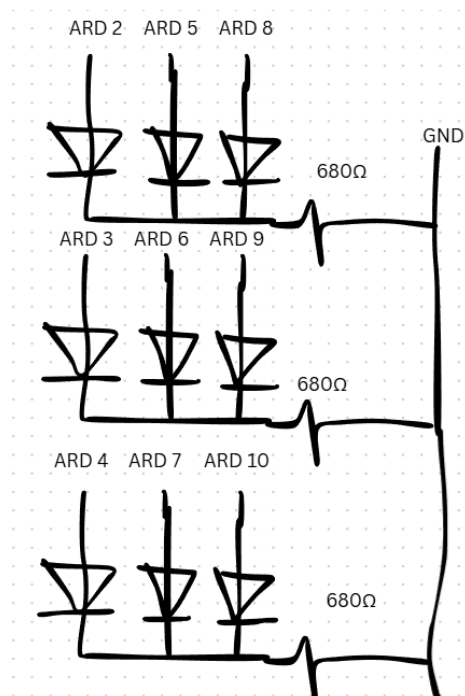


Fig. 6. Wiring diagram for 3x3 LED Matrix

## APPENDIX C

### TEST ARDUINO SKETCH

```

// predefined ship position (change this to test different positions)
int myShipPos = 6; // LED pin 6
int receivedTarget;
bool selectionPhase = true;
bool shootingPhase = false;
bool waitingForTarget = true;
int opponentScore = 0;

void setup(){
  Serial.begin(9600);
}

void loop(){
  if (selectionPhase){
    selectionPhase = false;
    shootingPhase = true;
    waitingForTarget = true;
  }
  if (shootingPhase && waitingForTarget){
    // wait for incoming target messages
    if (Serial.available()){
      receivedTarget = Serial.parseInt();
      while (Serial.available() > 0) {
        char c = Serial.read();
        if (c == '\n') break;
      }
      if (receivedTarget > 0){
        waitingForTarget = false;
        if(receivedTarget == 99){
          opponentScore = 0;
          selectionPhase = true;
          shootingPhase = false;
          waitingForTarget = true;
          return;
        }
        if(receivedTarget == myShipPos){
          opponentScore++;
          Serial.println(1);
          if(opponentScore >= 2){
            opponentScore = 0;
            selectionPhase = true;
            shootingPhase = false;
          }
          else{
            selectionPhase = true;
            shootingPhase = false;
          }
        }
        else{
          Serial.println(0);
        }
      }
    }
  }
}

```

## APPENDIX D FULL SYSTEM INTEGRATION

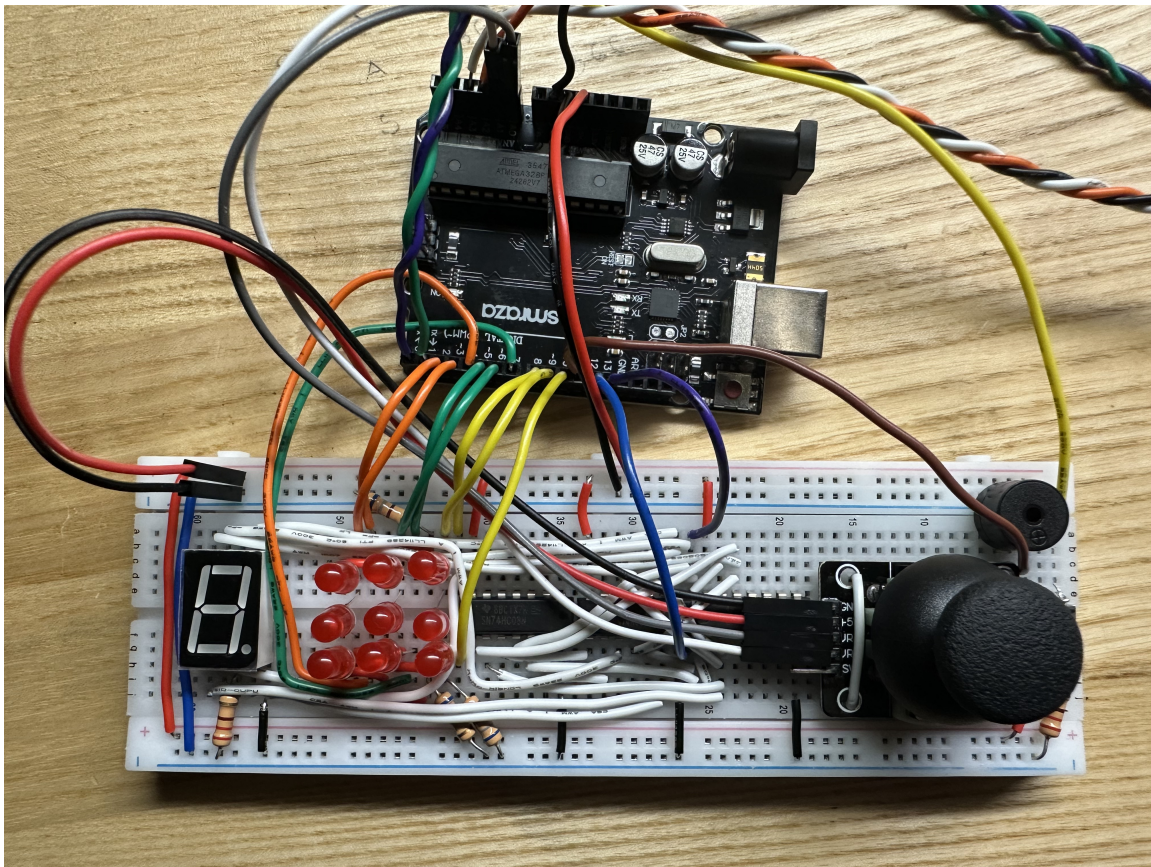


Fig. 7. View of Complete Controller with all Peripherals Wired

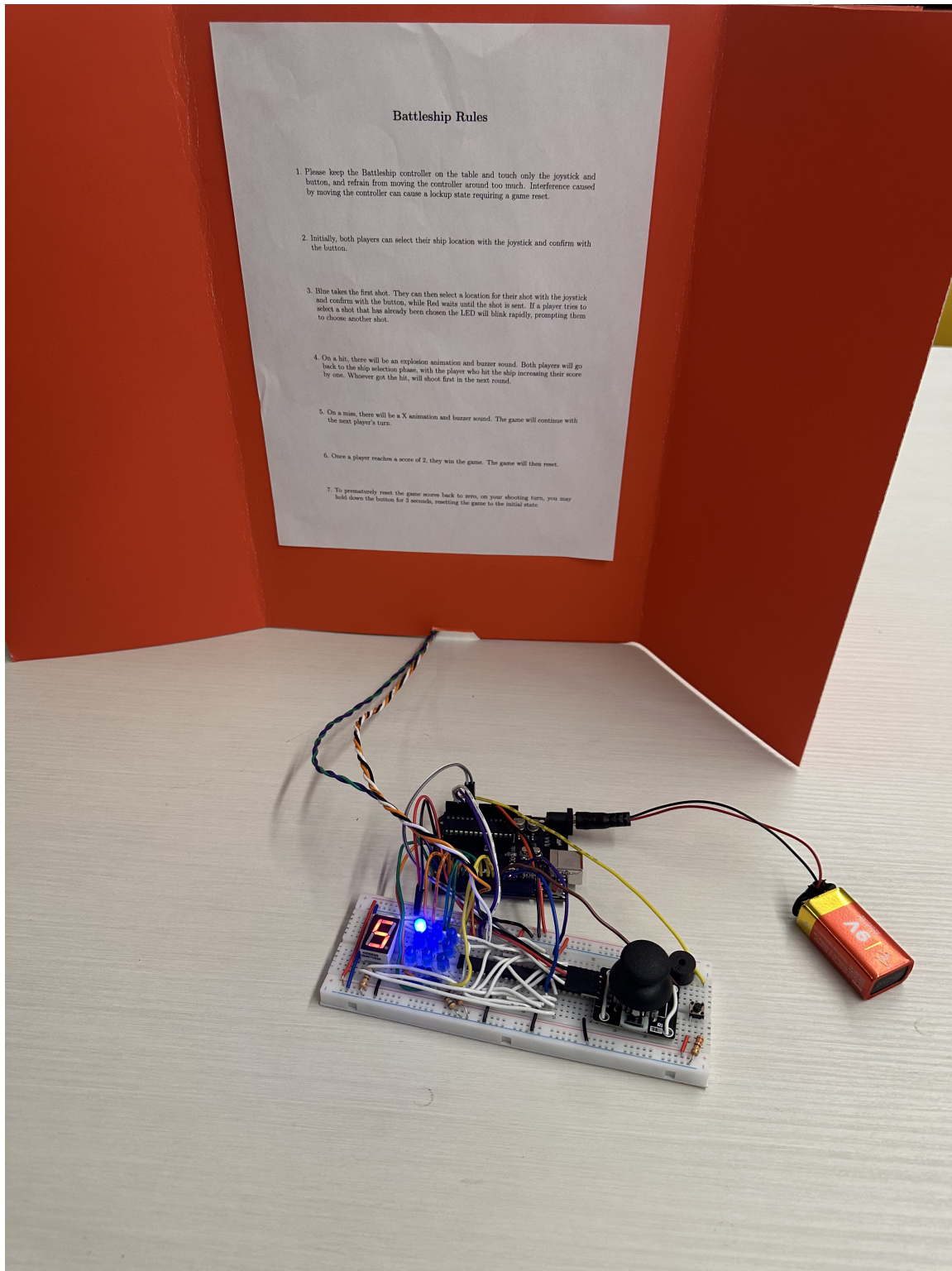


Fig. 8. Battery Powered Battleship Controller with Privacy Folder

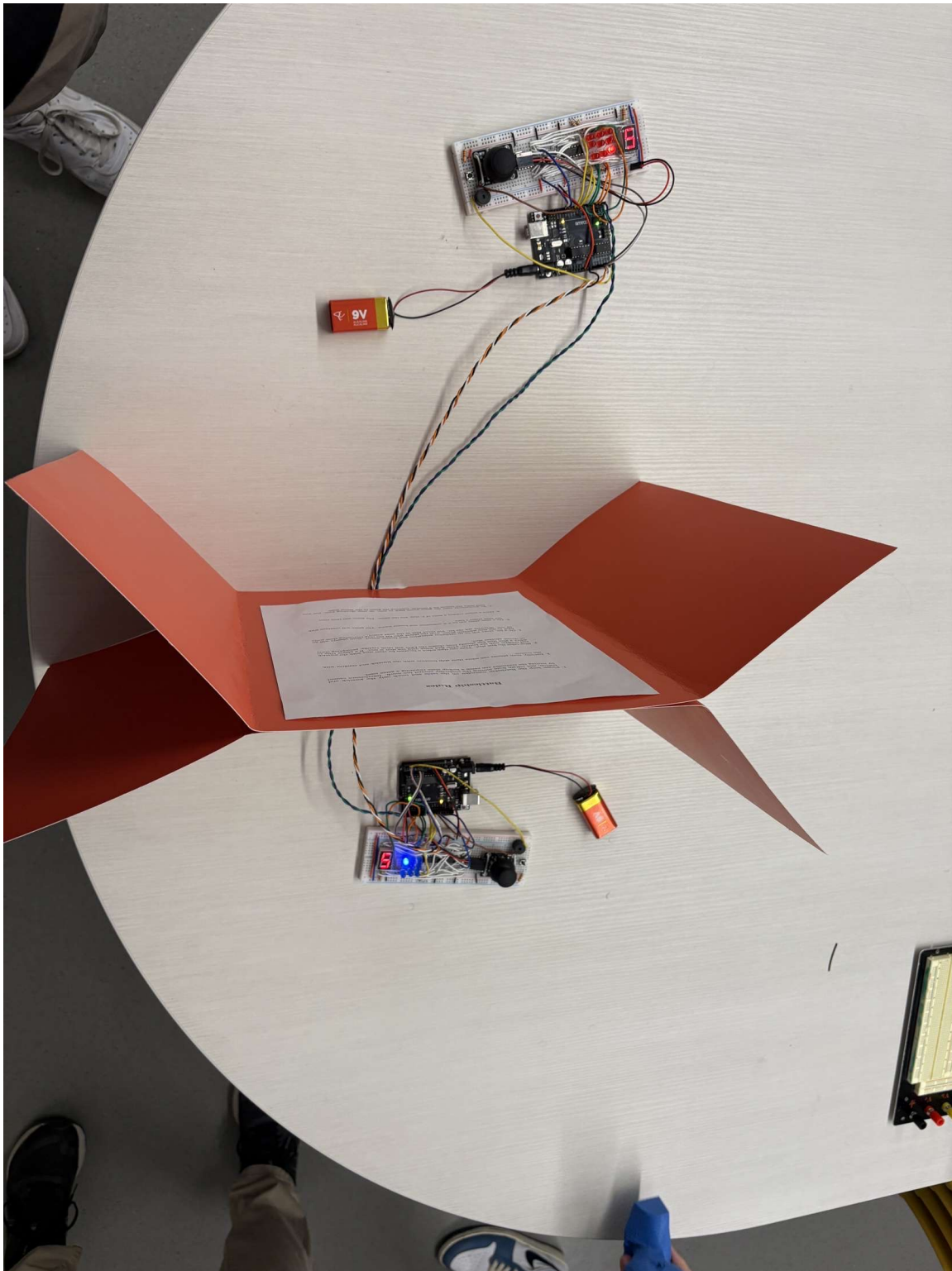


Fig. 9. Dual Controller Integration with Arduinos Connected Via Wiring Harness